

Real-Time Scheduling and OS Case Studies

Week 7: schedulability, fairness, and dispatching

SDB

Autumn 2026

Agenda: Real-Time & Modern OS Scheduling

Beyond General Purpose: Predictability and Fairness in Complex Systems

- | | |
|-----------------------------------|---|
| 1 Real-time models, RMS, and EDF | <i>(schedulability and guarantees)</i> |
| 2 Linux: CFS to EEVDF | <i>(fairness and latency)</i> |
| 3 Android case study | <i>(frame timing and worker priorities)</i> |
| 4 Windows Scheduler & Dispatching | <i>(responsiveness and priority management)</i> |
| 5 macOS case study | <i>(QoS and energy-aware work)</i> |
| 6 Comparison & Hands-on | |

Agenda: Real-Time & Modern OS Scheduling

Beyond General Purpose: Predictability and Fairness in Complex Systems

- | | |
|-----------------------------------|---|
| 1 Real-time models, RMS, and EDF | <i>(schedulability and guarantees)</i> |
| 2 Linux: CFS to EEVDF | <i>(fairness and latency)</i> |
| 3 Android case study | <i>(frame timing and worker priorities)</i> |
| 4 Windows Scheduler & Dispatching | <i>(responsiveness and priority management)</i> |
| 5 macOS case study | <i>(QoS and energy-aware work)</i> |
| 6 Comparison & Hands-on | |

Think Ahead: When Predictability is Paramount

What assumptions make a deadline guarantee meaningful? How do fixed-priority and deadline-driven policies differ, and how do general-purpose systems trade proportional fairness against latency?

Real-time scheduling: correctness includes timeliness

A real-time system must produce a logically correct result *and* produce it before its deadline. The scheduler is only one part of that guarantee.

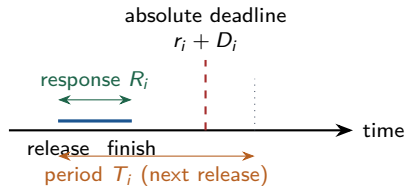
- **Hard:** a missed deadline is unacceptable under the stated fault model; design for bounded worst-case response.
- **Firm:** a late result has no value, but occasional misses may be tolerated.
- **Soft:** late results retain value; quality or utility degrades with lateness.

Guarantee boundary

A schedulability proof is conditional on a workload model, WCET bounds, release jitter, blocking, interrupt and dispatch costs, platform behavior, and the assumed fault model. A scheduler name alone is not a guarantee.

Periodic task model and timing vocabulary

- T_i : minimum inter-arrival time / period of task τ_i .
- C_i : worst-case execution demand of one job.
- D_i : relative deadline; absolute deadline is release time plus D_i .
- J_i : release jitter; B_i : maximum blocking from lower-priority work.
- R_i : worst-case response time from release to completion.



Utilization for implicit-deadline periodic tasks is

$$U = \sum_i C_i / T_i.$$

Basic uniprocessor results below assume independent, preemptible jobs, bounded execution, and negligible overhead unless a slide says otherwise.

Rate-monotonic scheduling: fixed-priority optimality

Priority assignment

For periodic tasks with $D_i = T_i$, assign higher fixed priority to shorter periods:

$$T_i < T_j \implies \tau_i \text{ has higher priority than } \tau_j.$$

Classical model:

- single preemptive processor; independent periodic or sporadic tasks;
- known WCET C_i , implicit deadlines $D_i = T_i$, no self-suspension;
- no blocking or overhead in the elementary utilization theorem.

Liu–Layland sufficient utilization bound:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1).$$

Interpretation

Passing certifies schedulability in this model. Failing this sufficient bound is *inconclusive*, not proof of a missed deadline. As $n \rightarrow \infty$, the bound tends to $\ln 2 \approx 0.693$.

For constrained deadlines $D_i \leq T_i$, shorter relative deadline gives higher priority under Deadline Monotonic (DM); do not automatically apply RMS ordering when $D_i \neq T_i$.

Fixed-priority analysis: response-time recurrence

When utilization tests are inconclusive, compute each task's worst-case response time in priority order. In the basic model:

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j, \quad R_i^{(0)} = C_i.$$

A task is schedulable if the recurrence converges to $R_i \leq D_i$ for every task. With blocking or jitter, the recurrence must include the corresponding terms.

A set the utilization bound does not certify

$\tau_1 = (C_1, T_1) = (1, 2)$ and $\tau_2 = (1, 3)$ give $U = 5/6 \approx 0.833$, just above the $n = 2$ bound 0.828. For τ_2 , iteration gives $1 \rightarrow 2 \rightarrow 2$, so $R_2 = 2 \leq 3$; both tasks meet deadlines in this model.

RMS example: utilization test and schedule

Implicit-deadline tasks

$$(C_1, T_1) = (1, 4), \quad (C_2, T_2) = (2, 6).$$

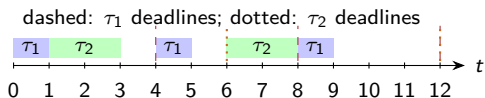
Shorter-period task τ_1 has higher fixed priority.

$$U = \frac{1}{4} + \frac{2}{6} = 0.583.$$

$$U_{LL}(2) = 2(\sqrt{2} - 1) \approx 0.828.$$

Since $U \leq U_{LL}$, the sufficient test certifies schedulability under the classical assumptions.

The schedule is illustrative; the utilization bound, not this finite trace, is the guarantee.



Earliest-deadline-first scheduling

At each scheduling decision, run the ready job with the earliest **absolute deadline**. A newly released job may preempt the running job when its deadline is earlier.

Exact utilization test: a special case

On one preemptive processor, for independent periodic/sporadic tasks with known WCET, no overhead or blocking, and **implicit deadlines** $D_i = T_i$:

$$\sum_i \frac{C_i}{T_i} \leq 1 \iff \text{EDF is schedulable.}$$

- This is a necessary-and-sufficient utilization test only for that implicit-deadline model.
- Under overload, EDF can miss deadlines; the miss pattern depends on the arrivals and workload.
- For other deadline models, use demand-based analysis rather than treating $U \leq 1$ as sufficient.

EDF with constrained deadlines: demand-bound test

For periodic or sporadic tasks with $D_i \leq T_i$, a job-demand view captures deadline density:

$$\text{dbf}_i(t) = \max\left(0, \left\lfloor \frac{t - D_i}{T_i} \right\rfloor + 1\right) C_i.$$

In the ideal preemptive uniprocessor model, EDF is feasible iff for every interval length $t > 0$:

$$\sum_i \text{dbf}_i(t) \leq t.$$

Why utilization alone can fail

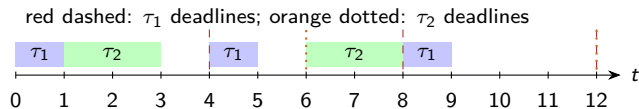
A short relative deadline can concentrate execution demand early even when average utilization $\sum_i C_i/T_i$ is below one. The demand-bound function asks whether any interval contains more work due than the processor can execute.

EDF example: deadline order and idle intervals

Implicit-deadline tasks: $\tau_1 = (C_1, T_1) = (1, 4)$ and $\tau_2 = (C_2, T_2) = (2, 6)$.

$$U = \frac{1}{4} + \frac{2}{6} = \frac{7}{12} \approx 0.583 < 1.$$

Both tasks are schedulable by the implicit-deadline EDF theorem under the stated ideal model.



The chart is one valid schedule; it is not the schedulability proof. At equal absolute deadlines, either tie-break is valid here.

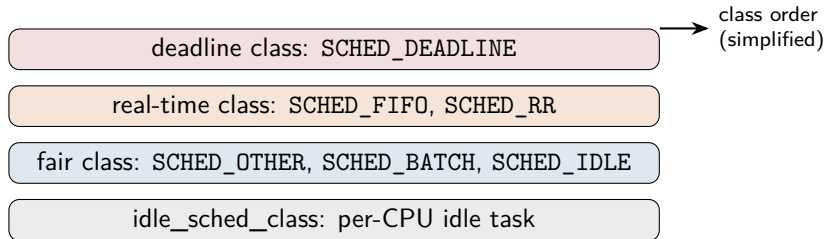
RMS vs EDF: A Direct Comparison

Property	Rate monotonic (fixed priority)	EDF (dynamic priority)
Priority key	Shorter period gets higher fixed priority	Earlier absolute deadline gets higher priority
Classical exact/sufficient test	Response-time analysis; LL utilization bound is sufficient only	$U \leq 1$ iff feasible for implicit deadlines; DBF for constrained deadlines
Overload	Fixed priority is stable/predictable by priority, but low-priority jobs can miss	Earliest-deadline order is optimal in the ideal uniprocessor model; overload still causes misses
Practical caveat	Blocking, jitter, and execution overhead must be modeled	Same; multiprocessor guarantees differ from uniprocessor results

Both rows compare idealized preemptive uniprocessor models; neither method makes WCET or blocking disappear.

Linux scheduling classes: policy is not one algorithm

Linux schedules runnable **threads/tasks**, not abstract processes. The kernel organizes scheduling policies into classes; higher-class runnable work can preempt lower-class work.



- `nice` adjusts fair-class weight; `SCHED_IDLE` is a fair policy, distinct from the CPU idle-task class.
- Run queues are per-CPU; affinity, migration, load balancing, and CPU topology affect observed behavior.
- The normal fair class targets proportional sharing and latency—not hard-deadline certification.

Class names and internals are Linux implementation details; consult documentation for the target kernel version.

Linux fair scheduling: from CFS to EEVDF

CFS: useful historical model

Track normalized service as virtual runtime. Conceptually, the runnable entity with the smallest virtual runtime is most under-served and becomes eligible to run.

- Run-queue ordering historically used a red-black tree.
- This remains useful for understanding weight-based proportional fairness.

EEVDF: modern fair-scheduler direction

Lag represents weighted ideal service minus actual service: positive lag means an entity is owed CPU. Select the earliest virtual deadline among eligible entities (nonnegative lag).

- A virtual deadline is an ordering key, not the application's real-time deadline.
- Requested slice length participates in virtual-deadline ordering, exposing latency/throughput trade-offs.

Interpretation

Do not teach “the current Linux scheduler always picks the leftmost $v_{runtime}$ node” as a version-independent fact. CFS is the conceptual predecessor; current mainline fair scheduling uses EEVDF mechanisms, and deployed kernels can differ by version/configuration.

Fair-class accounting: why weights matter

A simplified CFS-style accounting relation is

$$\Delta v_i \approx \Delta t_i \frac{W_0}{w_i}, \quad W_0 = 1024 \text{ (nice-0 reference weight)}.$$

Larger weight w_i means virtual service grows more slowly for the same execution time, so the entity receives a larger long-run share.

Runnable task	Relative weight	Idealized share of two-task CPU
nice 0	1024	$1024/(1024 + 335) \approx 75.3\%$
nice +5	335	$335/(1024 + 335) \approx 24.7\%$

- Shares are approximate long-run proportions among runnable peers on the same fair run queue.
- Sleep, cgroup hierarchy, CPU affinity, quota, competing classes, and multicore placement change the realized share.

Real-time policies in Linux: reservation versus priority

Policy	Dispatch rule (simplified)	Main risk / use
SCHED_FIFO	Fixed RT priority; run until block, yield, or preempted by higher priority	Starvation if a high-priority thread fails to block
SCHED_RR	FIFO among same-priority threads with a time quantum	Quantum is not a deadline guarantee
SCHED_DEADLINE	EDF-like deadlines with Constant Bandwidth Server reservation	Admission control and runtime/period constraints required

- Deadline scheduling parameters are commonly described as runtime Q , relative deadline D , and period T .
- A reservation limits interference from a task; it does not bound device, firmware, interrupt, or application WCET by itself.
- Privilege, resource limits, kernel configuration, and admission control govern whether a request is accepted.

Engineering rule

Never run an unbounded busy loop at elevated real-time priority on a production system.

Why uniprocessor schedulability does not transfer to multicore

Partitioned scheduling

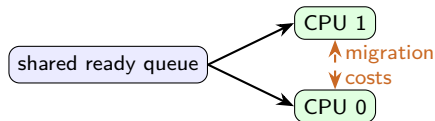
Assign each task to one CPU; schedule locally.

- Simple affinity and local analysis.
- Bin-packing can leave unusable capacity on one core while another is overloaded.

Global scheduling

One shared ready set; migrate jobs among CPUs.

- Can improve load balance.
- Migration, cache warmth, contention, and global analysis complicate bounds.



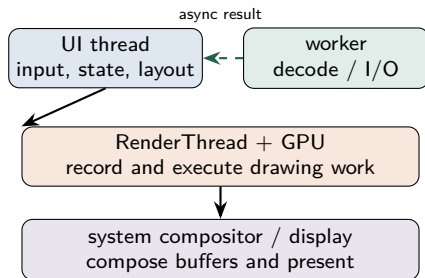
Global EDF and partitioned EDF have different feasibility properties; $U \leq m$ is not a universal multiprocessor feasibility test.

Case study: Android frame timing and background work

- At 60 Hz, a refresh is about 16.7 ms. App work must finish early enough for rendering and composition; this is a budget, not a reserved CPU slice.
- Keep input and frame-state updates short on the UI thread. Move decoding and I/O to bounded workers.
- Use `Process.setThreadPriority` conservatively; background priority suits deferrable worker work.
- Excessive priority can contend with UI/RenderThread and increase dropped frames. More threads do not create CPU capacity.

What the case teaches

Frame timing is end-to-end: app, GPU, compositor, and device load all matter. Priority is not a frame-deadline reservation.



Simplified pipeline; work can overlap, and details vary by API/device.

Windows scheduling: priority, quantum, and dynamic boosts

Windows selects runnable **threads** by priority. The documented priority space has 32 levels: level 0 is reserved for the zero-page thread; variable priorities occupy 1–15; real-time priorities occupy 16–31.

- Process priority class and thread-relative priority determine a thread's base priority.
- Threads at higher priority preempt lower-priority runnable threads; equal-priority threads share processor time according to quantum/ready-queue policy.
- The variable class (base priority 1–15) can receive temporary boosts for selected events, such as waking after I/O; boosts decay toward base priority.
- Real-time-class priorities are not dynamically boosted in the same way and can starve ordinary work if misused.

Correction to a common simplification

Windows is priority-driven, but calling it simply an “MLFQ” or saying every thread is preempted at quantum expiration hides important details. A quantum expiration may rotate equal-priority work; the dispatcher still considers ready-thread priorities and events.

Case study: macOS Quality of Service (QoS)

Classify work by its importance to the user:

- **User-interactive:** event handling and animation.
- **User-initiated:** a result the user is waiting for.
- **Utility:** long visible work, such as indexing.
- **Background:** deferrable maintenance or synchronization.

Example

Keep interaction on the main queue; index at utility QoS and defer non-urgent sync to background QoS.

user-interactive: UI/event handling

user-initiated: prompt result

utility: sustained visible work

background: deferrable work



lower urgency

```
let q = DispatchQueue(label: "indexer",  
                      qos: .utility)  
q.async { rebuildSearchIndex() }
```

Interpretation

QoS communicates intent to system frameworks and influences scheduling and resource policy; it is not a fixed CPU share or a hard deadline guarantee. Higher-priority work can also consume more energy.

Cross-platform lesson: policy is not a deadline contract

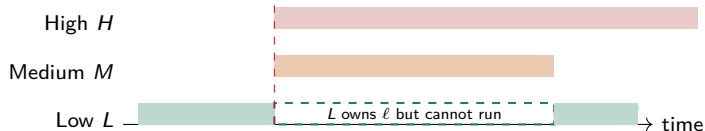
System	Application-facing control	What it helps express	What it does not prove
Linux	scheduler policy, nice, affinity, cgroups	class, relative share, placement/resource limits	end-to-end completion by an app deadline
Windows	process class and thread-relative priority	relative urgency and interactive responsiveness	bounded response under arbitrary load
Android	framework-managed threads and per-thread priority	UI/render urgency versus deferrable app work	stable frame timing across devices and thermal states
macOS	QoS on dispatch queues and tasks	importance, latency/throughput, and energy intent	reserved CPU capacity or hard real-time behavior

Engineering workflow

State the latency target, bound the work, avoid blocking critical paths, measure tail latency under contention, and define overload behavior before tuning priority.

Names and implementation details are release- and platform-dependent. Treat developer-facing priorities as policy inputs, not portable timing guarantees.

Priority inversion: a scheduling failure mediated by locks



A low-priority lock holder can indirectly delay a high-priority task while medium-priority work preempts the holder.

- **Priority inheritance:** temporarily raise the holder's priority until it releases the resource.
- **Priority ceiling:** protocol-specific ceiling rules bound blocking and prevent certain deadlocks.
- Neither protocol removes the need to bound critical-section duration and nested locking.

Hands-on Linux: inspect scheduler behavior

- Inspect per-thread policy, priority, nice value, state, and last CPU.
- Compare nice weights only under comparable fair-class workloads; affinity, cgroups, quotas, and other runnable work affect measured shares.

Per-thread view (Linux procps)

```
ps -eLo pid,tid,cls,rtprio,pri,ni,psr,stat,comm
```

Inspect policy/parameters for one process or thread

```
chrt -p "$PID"
```

Short-lived CPU-bound comparison; stop both jobs afterward

```
yes >/dev/null & a=$!
```

```
nice -n 10 yes >/dev/null & b=$!
```

```
sleep 5
```

```
ps -p "$a,$b" -o pid,ni,cls,pri,pcpu,comm
```

```
kill "$a" "$b"
```

```
wait "$a" "$b" 2>/dev/null
```

Optional, if installed/permitted: use `perf sched record` and `perf sched timehist` to inspect scheduling delays. Kernel debug interfaces vary by configuration and version.

Key Takeaways

- Deadline claims require a workload/platform model and analysis; a scheduler label alone is not a guarantee.
- RMS is fixed-priority: its Liu–Layland utilization test is sufficient, while response-time analysis can certify additional task sets.
- EDF's $U \leq 1$ test is exact for ideal implicit-deadline uniprocessor tasks; constrained deadlines require demand analysis.
- Linux fair scheduling evolved from CFS accounting toward EEVDF; policy, per-CPU placement, and cgroups matter.
- Windows dispatches threads by priority; dynamic boosts are conditional and version/API context matters.
- Android frame pacing and macOS QoS both require matching work importance to the user-visible latency and energy goal.
- Blocking, overload, multicore interference, and WCET uncertainty often dominate the scheduler choice.

Reflection Prompt: The Scheduler's Dilemma

For low-latency audio sharing a multicore phone with a CPU-heavy upload, specify the workload model, interference budget, admission policy, and fallback behavior before choosing a scheduler.

Next Week Preview: Threads and Multithreading Models

From Processes to Threads: Finer-Grained Concurrency

- **Introduction to Threads:** Why threads are needed, process vs. thread.
- **User-Level vs. Kernel-Level Threads:** Understanding their differences and implementation.
- **Multithreading Models:** One-to-One, Many-to-One, Many-to-Many models.
- **Thread Libraries ('pthreads'):** How developers create and manage threads.
- **Thread Context Switching:** How it differs from process context switching.

Next Week Preview: Threads and Multithreading Models

From Processes to Threads: Finer-Grained Concurrency

- **Introduction to Threads:** Why threads are needed, process vs. thread.
- **User-Level vs. Kernel-Level Threads:** Understanding their differences and implementation.
- **Multithreading Models:** One-to-One, Many-to-One, Many-to-Many models.
- **Thread Libraries ('pthreads'):** How developers create and manage threads.
- **Thread Context Switching:** How it differs from process context switching.

Prep Tip for Next Session

Review the concept of a process Control Block (PCB) and process context. Think about what information would still be unique to a thread and what could be shared among threads within the same process.

Quick Quiz: Real-Time & Modern Scheduling I

Test Your Conceptual Understanding:

- ① **Scenario:** Two implicit-deadline periodic tasks: A ($C = 2, T = 5$) and B ($C = 1, T = 3$).
 - Which task has higher RMS priority? Compute U and compare with the $n = 2$ bound.
 - What does the implicit-deadline EDF test conclude? Which result is sufficient versus exact?
- ② **True/False:** Two continuously runnable fair-class threads with nice 0 and nice +10 receive equal CPU shares on one otherwise idle CPU. State your assumptions.
- ③ **Define:** What is a temporary Windows dynamic-priority boost intended to improve, and why is it not a hard real-time guarantee?
- ④ **Application:** When is uniprocessor EDF utilization analysis insufficient for a multicore or constrained-deadline workload?

Quick Quiz: Real-Time & Modern Scheduling II

Think & Discuss

Formulate your answers before reviewing the solutions or discussing with peers.

Exercise: Real-Time Schedulability Analysis I

Part 1: RMS Schedulability Check

Tasks: (Execution Time C_i , Period T_i)

- Task A: $C_A = 2$, $T_A = 5$
- Task B: $C_B = 1$, $T_B = 4$
- Task C: $C_C = 1$, $T_C = 10$

Task:

- 1 Calculate the total CPU utilization (U) for this set of tasks.
- 2 Calculate the RMS schedulability bound for $n = 3$ tasks.
- 3 Based on the RMS schedulability test, can these tasks be guaranteed to be scheduled by RMS?
Show your calculations.
- 4 What are the RMS priorities for Task A, B, and C?

Exercise: Real-Time Schedulability Analysis II

Part 2: Conceptual Comparison

Scenario: A new game console OS needs to be designed. It handles high-priority game rendering processes, background downloads, and occasional real-time voice chat.

Task:

- 1 Would a pure FCFS scheduler be suitable? Why or why not?
- 2 Would a pure EDF scheduler be a good choice for **all** tasks? Discuss potential benefits and drawbacks in this mixed environment.
- 3 Propose a hybrid scheduling approach (e.g., using concepts from MLQ, Priority, and fairness) that could effectively manage these diverse workloads. Justify your proposal.

Reminder

Remember to clearly state your assumptions and show all steps for calculations.

Advanced Topics to Explore I

Delving Deeper into Real-Time and Advanced Schedulers

I. Real-Time Scheduling Extensions

- **Resource Sharing in RTOS:** Understanding solutions like Priority Inheritance Protocol (PIP) and Priority Ceiling Protocol (PCP) to prevent priority inversion.
- **Demand and response-time analysis:** dbf tests for EDF, response-time analysis with blocking and release jitter for fixed priority.
- **Aperiodic service:** deferrable/sporadic servers and reservation-based bandwidth isolation.
- **Multicore and mixed criticality:** partitioning, global scheduling, migration costs, and mode-change analysis.

Advanced Topics to Explore II

II. Advanced Aspects of General-Purpose Schedulers

- **Linux fair scheduling:** EEVDF eligibility/virtual deadlines, cgroup CPU weights and quotas, per-CPU run queues, and load balancing.
- **Windows:** base/dynamic priority, boosts, processor affinity, and processor groups (details depend on Windows release and API).
- **Scheduling interfaces:** compare Linux `sched_setscheduler/sched_setattr/setpriority` with Windows `SetThreadPriority`; inspect privilege and admission constraints.

Advanced Topics to Explore III

III. Emerging & Specialized Scheduling Areas

- **Energy and heterogeneous CPUs:** DVFS, capacity-aware placement, thermal constraints, and big.LITTLE scheduling.
- **Virtualization and nested scheduling:** how guest vCPUs compete with host tasks and how steal time affects deadlines.
- **Alternative policies:** lottery/stride proportional share, gang scheduling, GPU scheduling, and network packet schedulers.

Primary references and version notes

- Linux kernel, *EEVDF Scheduler*: <https://docs.kernel.org/scheduler/sched-eevdf.html>
- Linux kernel, *CFS Scheduler*: <https://docs.kernel.org/scheduler/sched-design-CFS.html>
- Linux man-pages, *sched(7) scheduling policies*: <https://man7.org/linux/man-pages/man7/sched.7.html>
- Linux kernel, *Deadline Task Scheduling*: <https://docs.kernel.org/scheduler/sched-deadline.html>
- Microsoft Learn, *Scheduling Priorities*:
<https://learn.microsoft.com/en-us/windows/win32/procthread/scheduling-priorities>
- Microsoft Learn, *Priority Boosts*:
<https://learn.microsoft.com/en-us/windows/win32/procthread/priority-boosts>
- Android Developers, *Better performance through threading*:
<https://developer.android.com/topic/performance/threads>
- Android Developers, *Process.setThreadPriority*:
<https://developer.android.com/reference/android/os/Process>
- Apple Developer, *DispatchQoS*: <https://developer.apple.com/documentation/dispatch/dispatchqos>
- C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *JACM*, 20(1), 1973, pp. 46–61.

Version discipline

Scheduler internals and platform APIs change. CFS is retained as a historical/accounting model; consult target Linux, Android, Windows, and macOS documentation before making implementation-specific claims.